

UNIVERZITET U NIŠU

ELEKTRONSKI FAKULTET

KVALITET I TEHNIKE PREDOBRADE TEKSTUALNIH PODATAKA NA PRIMERU KLASIFIKACIJE TVITOVA

SEMINARSKI RAD

Predmet:

Prikupljanje i predobrada podataka za mašinsko učenje

Mentor:
prof. dr Aleksandar Stanimirović

Student:
Jovan Paunović 2122

Niš, mart 2026. godina

SADRŽAJ

1. UVOD 4
2. KVALITET TEKSTUALNIH PODATAKA 5
2.1. Dužina dokumenata 5
2.2. Veličina vokabulara 6
2.3. Leksička raznovrsnost 6
2.4. Šum u tekstu 7
3. PREDOBRADA TEKSTUALNIH PODATAKA 9
3.1. Uklanjanje duplikata 9
3.2. Uklanjanje šuma 10
3.2.1. Identifikacija šuma 10
3.2.2. Tehnike uklanjanja šuma 10
3.3. Tokenizacija 11
3.4. Normalizacija teksta 12
3.4.1. Normalizacija veličine slova (*case folding*) 12
3.4.2. *Unicode* normalizacija 13
3.4.3. Proširivanje skraćenica 14
3.4.4. Normalizacija *emoji* simbola 14
3.4.5. Korekcija pravopisa 15
3.5. Uklanjanje stop reči 15
3.6. Lematizacija i stemovanje 16
3.6.1. Stemovanje 17
3.6.2. Lematizacija 17
3.6.3. Poređenje tehnika 17
3.7. Augmentacija tekstualnih podataka 18
3.7.1. Jednostavna augmentacija tekstualnih podataka 19
3.7.2. Augmentacija prevođenjem 19
3.7.3. Generativni modeli 19
3.7.4. Prednosti i mane augmentacije tekstualnih podataka 19
4. UTICAJ PREDOBRADE NA METODE REPREZENTACIJE TEKSTA 20
4.1. Klasične metode zasnovane na frekvencijama reči (BoW i TF-IDF) 20
4.2. Vektorske reprezentacije reči (*Word2Vec*, *GloVe* i *FastText*) 22

4.3. Transformeri (BERT i srodni modeli) 23
5. ZAKLJUČAK 24
LITERATURA 25

1. UVOD

Sa razvojem digitalnih platformi za komunikaciju, količina korisnički generisanog tekstualnog sadržaja beleži eksponencijalni rast. Ovi podaci predstavljaju bogat izvor informacija za različite analitičke zadatke kao što su analiza sentimenta, detekcija govora mržnje ili klasifikacija sadržaja po tipu. Međutim, sirovi tekst generisan od strane korisnika često je neuređen, pun šuma (engl. *noise*) i nekonzistentan – što predstavlja značajan izazov za računarske metode obrade prirodnog jezika (engl. *Natural Language Processing, NLP*). Takav tekst karakterišu skraćenice, emotikoni, sleng, greške u pisanju, specijalni karakteri i neformalna gramatika, što otežava direktnu primenu algoritama mašinskog učenja bez prethodne obrade.

Upravo zbog ovog šuma u podacima, kvalitet tekstualnih podataka postaje ključna komponenta uspešnosti modela za klasifikaciju. Kvalitet podataka u ovom kontekstu obuhvata stepen u kojem tekst sadrži relevantne i dosledne informacije koje algoritmi mogu da nauče, uz najmanje moguće ometajuće elemente. Ako podaci sadrže previše nerelevantnih karaktera ili nesistematičnih obrazaca, modeli mogu naučiti obrasce šuma umesto značenja samog teksta, što vodi do loših performansi pri klasifikaciji.

Da bi se povećala efikasnost i tačnost modela, koristi se niz tehnika predobrade tekstualnih podataka. Predobrada obuhvata korake kao što su čišćenje teksta, tokenizacija, normalizacija, uklanjanje nepotrebnih elemenata (npr. stop reči) i transformacija reči u numeričke reprezentacije pogodne za modele. Ove tehnike služe da redukuju šum, standardizuju format teksta i omogućavaju modelima da efikasnije uče relevantne uzorke iz podataka.

U ovom radu biće predstavljene teorijske osnove kvaliteta tekstualnih podataka i tehnika predobrade koje se obično koriste u mašinskom učenju i NLP-u. Takođe, biće obrađeni principi i izazovi povezani sa predobradom teksta i njihov uticaj na performanse klasifikacionih modela. Na kraju, biće sažeto prikazana primena ovih tehnika u praktičnom delu rada, uz naglasak na to kako one doprinose poboljšanju rezultata u procesu klasifikacije tekstualnih podataka.

2. KVALITET TEKSTUALNIH PODATAKA

Klasične dimenzije kvaliteta podataka – tačnost, potpunost, konzistentnost i pravovremenost – razvijene su prevashodno u kontekstu strukturiranih podataka sa jasno definisanim šemama i domenima. Kada se iste mere primene na nestrukturirane tekstualne podatke, njihova interpretacija se suštinski menja: ne postoji referentna šema prema kojoj bi se proveravala ispravnost vrednosti, niti strogo definisano polje koje može biti "prazno" u klasičnom smislu. Kvalitet podataka je uvek kontekstualno uslovljen, a ovo važi posebno za tekst, gde iste informacije mogu biti izražene na bezbroj načina, različitim rečnikom, dužinom i stilom [1].

Shodno tome, procena kvaliteta tekstualnih podataka zahteva skup mera prilagođenih prirodi jezičkog sadržaja. U nastavku su opisane ključne metrike koje se koriste u tu svrhu: dužina dokumenta, veličina vokabulara, leksička raznovrsnost, šum kao i drugi pokazatelji koji zajedno daju uvid u strukturu i upotrebljivost tekstualnih korpusa.

2.1. Dužina dokumenata

Dužina dokumenata jedna je od osnovnih i najlakše merljivih karakteristika tekstualnih podataka, a najčešće se izražava kao broj reči, broj karaktera ili broj tokena. Ona predstavlja jedan od osnovnih pokazatelja kvaliteta tekstualnih podataka i značajno utiče na performanse modela mašinskog učenja u obradi prirodnog jezika. Kratki dokumenti često sadrže ograničen kontekst i mogu imati nedovoljno informacija za efikasnu klasifikaciju ili generisanje semantičkih reprezentacija. S druge strane, kada se modeli primene na duže dokumente dolazi do problema – kontekst se skraćuje, semantika se gubi, a zadaci poput pronalaženja sličnosti postaju nepouzdani, što može dovesti do zasićenja modela i povećati potrebu za dodatnom predobradom, poput odsecanja ili sažimanja teksta.

Prilikom analize dužine dokumenata često se koriste statističke metrike kao što su prosečan broj tokena po dokumentu, medijana dužine, minimalne i maksimalne vrednosti, kao i distribucija dužine dokumenata unutar skupa podataka. Ove informacije pomažu u izboru odgovarajućih tehnika predobrade i podešavanja modela, na primer pri odlučivanju o maksimalnom broju tokena u sekvencama za modele zasnovane na *transformer*-ima.

Razumevanje dužine dokumenata važno je i za strategije augmentacije podataka i balansiranja skupa podataka, jer nejednaka dužina može uzrokovati pristrasnost modela prema dužim ili kraćim tekstovima. Efikasno upravljanje ovom karakteristikom doprinosi kvalitetnijim i robusnijim modelima u NLP zadacima [2].

Dužina dokumenata direktno utiče i na način reprezentacije teksta. Kod modela zasnovanih na BoW (engl. *Bag of Words*) i TF-IDF (engl. *Term Frequency-Inverse Document Frequency*) pristupu, duži dokumenti prirodno imaju veće frekvencije termina, što može dovesti do pristrasnosti ako se ne primeni odgovarajuća normalizacija. Kod *transformer* modela, ograničenje maksimalne dužine sekvence (npr. 512 tokena) zahteva pažljivo

skraćivanje ili segmentaciju teksta, jer u suprotnom dolazi do gubitka relevantnog konteksta. Zbog toga analiza dužine dokumenata predstavlja važan korak pre izbora i podešavanja modela.

2.2. Veličina vokabulara

Veličina vokabulara predstavlja ukupan broj jedinstvenih tokena (reči ili podreči) koji se pojavljuju u korpusu tekstualnih podataka. U kontekstu dubokog učenja za obradu prirodnog jezika, vokabular definiše skup reči koje model prepoznaje – sve reči van tog skupa se ignorišu ili tretiraju kao nepoznate prilikom obrade novih tekstova.

Ova metrika ima direktan uticaj na nekoliko ključnih aspekata modela. Pre svega, veličina vokabulara određuje dimenzije matrice ugrađivanja (engl. *embedding matrix*), čiji su redovi vektorske reprezentacije svake reči. Pored toga, veliki vokabular povećava memorijske zahteve i produžava vreme treniranja, dok istovremeno može unositi šum kroz retke reči za koje model nema dovoljno konteksta da nauči pouzdanu semantičku reprezentaciju. S druge strane, previše ograničen vokabular može dovesti do gubitka važnih informacija i slabije generalizacije modela.

Veličina vokabulara se određuje analizom distribucije frekvencija reči u skupu podataka. Budući da u prirodnom jeziku mali broj reči čini većinu pojavljivanja, standardni pristup je da se identifikuje koliko najfrekventnijih reči pokriva određeni procenat svih tokena u tekstu – najčešće 90%. Reči koje spadaju u tzv. *long-tail* distribuciju, odnosno koje se pojavljuju retko, odbacuju se jer ne doprinose kvalitetu naučenih reprezentacija. Na ovaj način postiže se balans između ekspresivnosti modela i računarske efikasnosti [3].

2.3. Leksička raznovrsnost

Leksička raznovrsnost je mera koja pokazuje koliko je bogat i raznovrstan rečnik u tekstu, odnosno u kojoj meri se reči ponavljaju nasuprot tome da se pojavljuju nove, različite reči. Visoka leksička raznovrsnost ukazuje na širok i varijabilan rečnik, dok niska raznovrsnost sugeriše da se relativno mali skup reči učestalo ponavlja kroz tekst.

Osnovna i najšire korišćena mera leksičke raznovrsnosti je odnos tipova i tokena, poznat kao TTR (engl. *Type-Token Ratio*). Računa se kao količnik broja jedinstvenih reči (tipova) i ukupnog broja reči (tokena) u tekstu:

$$TTR = \frac{broj\ jedinstvenih\ reči}{ukupan\ broj\ reči}$$

Vrednost TTR blizu 1 ukazuje na visoku raznovrsnost, gde se većina reči pojavljuje samo jednom, dok vrednost blizu 0 ukazuje na visoko ponavljanje istih reči kroz tekst. U

kontekstu obrade prirodnog jezika, nizak TTR karakterističan je za domenski specifične tekstove u kojima se isti termini stalno ponavljaju. Ovakva karakteristika skupa podataka može biti povoljna za zadatke klasifikacije teksta, jer model lakše uočava pouzdane obrasce koji razlikuju pojedine kategorije – svaka klasa teži da koristi specifičan i prepoznatljiv skup reči.

Leksička raznovrsnost ima i direktan uticaj na kvalitet naučenih reprezentacija. Visoka raznovrsnost znači veći vokabular, što od modela zahteva učenje većeg broja *embedding* vektora, od kojih mnogi imaju malo primera za treniranje. Nizak TTR, s druge strane, znači da se isti termini pojavljuju češće kroz skup podataka, što modelu pruža više konteksta za učenje pouzdanih reprezentacija tih reči. Međutim, ovakav dataset može biti ograničen u pogledu generalizacije na tekstove koji koriste drugačiji ili bogatiji rečnik od onog viđenog tokom treniranja [4].

Nedostatak TTR-a je njegova osetljivost na dužinu teksta. Prema Zipfovom zakonu, kako tekst postaje duži, prirodno je da se nove reči pojavljuju sve ređe, dok se osnovne reči kao što su veznici, predlozi i česti pojmovi neprestano ponavljaju. Zbog toga će duži skupovi podataka matematički uvek imati manji TTR od kraćih, čak i ako su leksički jednako bogati, što otežava pouzdano poređenje između kategorija različitih veličina.

Da bi se ovaj problem prevazišao, u praksi se često koristi standardizovani TTR (STTR). Za razliku od klasičnog TTR-a koji se računa nad celim tekstom, STTR deli tekst na segmente fiksne dužine (najčešće 1000 tokena), računa TTR za svaki segment zasebno, a zatim uzima prosečnu vrednost:

$$STTR = \frac{1}{n}\sum_{i=1}^{n} TTR_i$$

Na ovaj način poređenje leksičke raznovrsnosti između tekstova različitih dužina postaje metodološki opravdanije i pouzdanije.

2.4. Šum u tekstu

Šum u tekstu obuhvata sve elemente koji su irelevantni ili štetni za performanse modela obrade prirodnog jezika. Prisustvo šuma zamagljuje obrasce i značenje u tekstu, što direktno utiče na kvalitet tokenizacije, ekstrakcije karakteristika i naučenih reprezentacija. Budući da tekstualni podaci prikupljeni iz realnih izvora retko kada dolaze u čistom i strukturiranom obliku, identifikacija i uklanjanje šuma predstavlja jedan od ključnih koraka u pripremi podataka. Šum se može pojaviti u različitim oblicima:

- **Tipografske greške i pogrešno napisane reči** – model ih tretira kao potpuno nove, nepoznate tokene, što nepotrebno povećava veličinu vokabulara i uvodi retke tokene bez pouzdane semantičke reprezentacije,
- **HTML i XML oznake i ostaci *web scraping*-a** – URL-ovi, tagovi i specijalni karakteri nemaju semantičku vrednost i direktno ometaju obradu teksta,

- **Emoji i emotikoni** – iako mogu nositi semantičke informacije, remete standardnu tokenizaciju kada model nije prilagođen za njihovu obradu,
- **OCR greške** – nastaju pri automatskom prepoznavanju teksta sa slike i generišu karakteristične kombinacije karaktera koje ne odgovaraju nijednoj reči u vokabularu,
- **Nepravilni razmaci i nevidljivi karakteri** – višestruki razmaci, tabulatori i prelomi redova ometaju tokenizaciju,
- **Greške u kodiranju karaktera** – pogrešno dekodiran tekst rezultuje besmislenim simbolima koji se ne mogu smisleno interpretirati,
- **Metapodaci i standardni tekst (engl. *boilerplate*)** – zaglavlja, fusnote i automatski generisane poruke (npr. "*Unsubscribe | Privacy Policy | Terms of Service*" na kraju mejla) razređuju sadržaj i unose irelevantne statističke obrasce,
- **Namerni šum (engl. *obfuscation*)** – svesno maskiranje reči specijalnim simbolima (npr. unutar psovki) radi zaobilaženja filtera na društvenim mrežama; iako tehnički predstavlja šum, u klasifikaciji toksičnog teksta ovo je zapravo bitan prediktivni signal.

Prisustvo šuma ima višestruke negativne posledice po model. Pored nepotrebnog povećanja vokabulara, šum smanjuje kvalitet naučenih vektorskih reprezentacija, uvodi lažne statističke obrasce i otežava sve naredne korake u *pipeline*-u, od tokenizacije do ekstrakcije karakteristika.

Važno je napomenuti da je pojam šuma uvek zavisan od konkretnog zadatka. Na primer, brojevi mogu biti potpuno irelevantan šum u zadacima analize sentimenta, ali ključna informacija u zadacima ekstrakcije podataka ili prepoznavanja entiteta. Zbog toga odluku o tome šta se tretira kao šum treba doneti uzimajući u obzir prirodu problema koji se rešava [5].

3. PREDOBRADA TEKSTUALNIH PODATAKA

Predobrada tekstualnih podataka predstavlja skup tehnika kojima se sirovi tekst transformiše u oblik pogodan za dalju analizu i treniranje modela mašinskog učenja. Tekstualni podaci prikupljeni iz realnih izvora gotovo nikada nisu u čistom i strukturiranom obliku – sadrže šum, nedoslednosti, irelevantne elemente i različite varijante istih reči koje model ne može automatski da prepozna. Bez odgovarajuće predobrade, model uči na osnovu zašumljenih i nekonzistentnih podataka, što direktno utiče na kvalitet naučenih reprezentacija i performanse klasifikacije. U nastavku ovog poglavlja opisani su ključni koraci predobrade teksta: uklanjanje duplikata, tokenizacija, normalizacija teksta, lematizacija, stemovanje, uklanjanje stop reči, kao i tehnike augmentacije podataka.

3.1. Uklanjanje duplikata

Duplikati kod tekstualnih podataka predstavljaju identične ili gotovo identične zapise koji se pojavljuju više puta u skupu podataka. Njihovo prisustvo može značajno narušiti kvalitet treniranog modela, zbog čega je uklanjanje duplikata jedan od prvih koraka u procesu predobrade. Razlikuju se dve osnovne vrste duplikata:

- **Egzaktni duplikati** – potpuno identični tekstovi koji nastaju najčešće kao posledica grešaka pri prikupljanju podataka, višestrukog unosa ili spajanja različitih izvora. Identifikuju se poređenjem heš vrednosti tekstova, što je računarski jednostavna i brza operacija.
- ***Near*-duplikati** – tekstovi koji su sadržajno gotovo isti, ali se razlikuju u manjim detaljima poput tipografskih grešaka, različite interpunkcije ili neznatno izmenjenih formulacija. Zahtevaju mere sličnosti kao što su *Jaccard* sličnost, kosinusna sličnost ili *Levenshtein*-ova distanca. Za velike skupove podataka često se koristi *MinHash* algoritam koji omogućava efikasnu aproksimativnu detekciju bez poređenja svakog para tekstova.

Prisustvo duplikata ima višestruke negativne posledice:

- Model koji višestruko uči iste primere postaje pristrasan prema njima, što dovodi do *overfitting*-a i lošije generalizacije,
- Kada isti tekst postoji i u skupu za treniranje i u skupu za testiranje, dolazi do curenja podataka (engl. *data leakage*), pri čemu metrike evaluacije daju optimistične rezultate koji ne odražavaju stvarne performanse modela,
- Duplikati mogu iskriviti distribuciju klasa i narušiti statističke analize sprovedene nad skupom podataka.

Nakon identifikacije, uklanjanje duplikata se vrši zadržavanjem jednog primerka svakog jedinstvenog teksta, dok se ostali brišu iz skupa podataka [6], [7].

3.2. Uklanjanje šuma

U poglavlju o kvalitetu tekstualnih podataka opisane su vrste šuma koje se mogu pojaviti u tekstu. U ovom odeljku fokus je na konkretnim tehnikama kojima se taj šum identifikuje i uklanja pre daljeg procesiranja [5].

3.2.1. Identifikacija šuma

Pre samog uklanjanja, neophodno je identifikovati koje vrste šuma su prisutne u skupu podataka. Postoje tri osnovna pristupa:

- **Ručna inspekcija uzoraka** – za manje skupove podataka ili inicijalnu eksploraciju često najdirektnije sredstvo. Pregledom nekoliko stotina tekstova mogu se uočiti dominantni obrasci šuma.
- **Analiza frekvencija karaktera i tokena** – neuobičajeno visoka frekvencija određenih simbola ili kratkih tokena često ukazuje na prisustvo šuma.
- **Regularni izrazi** – omogućavaju preciznu detekciju strukturisanog šuma kao što su URL-ovi, HTML tagovi ili mejl adrese.

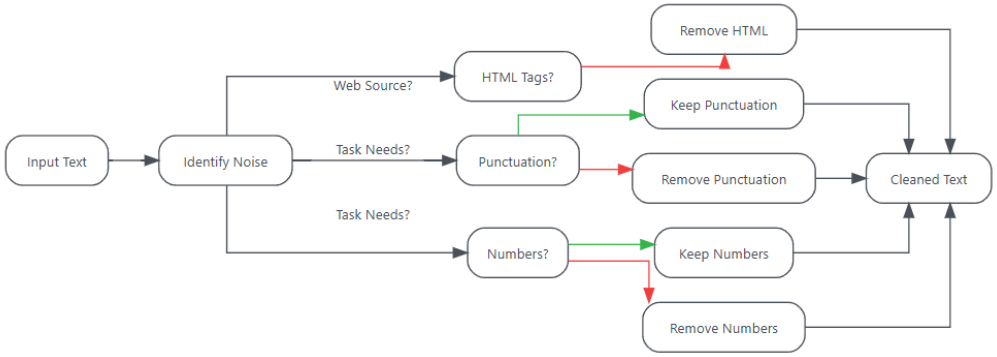
3.2.2. Tehnike uklanjanja šuma

U praksi se uklanjanje šuma najčešće implementira kao niz sekvencijalnih koraka u kojima se svaka vrsta šuma obrađuje zasebnim regularnim izrazom ili specijalizovanom bibliotekom. U zavisnosti od tipa šuma koriste se različite tehnike:

- **Uklanjanje HTML oznaka** – tekst prikupljen *web scraping*-om često sadrži HTML tagove koji nemaju semantičku vrednost. Za jednostavne slučajeve jednostavan regularni izraz je dovoljan, dok se za kompleksnije HTML strukture preporučuje korišćenje biblioteke *BeautifulSoup* koja je robusnija i pouzdanija pri parsiranju.
- **Uklanjanje URL-ova, @tagovanja i #heštegova** – ovi elementi se efikasno uklanjaju ili zamenjuju drugim tokenima (npr. URL → "url") pomoću regularnih izraza. Zamena drugim tokenom ponekad je bolja od potpunog uklanjanja jer model zadržava informaciju da je na tom mestu postojao link ili pominjanje korisnika.
- **Uklanjanje interpunkcije** – interpunkcija se u većini zadataka klasifikacije teksta uklanja jer ne doprinosi semantičkom sadržaju. Međutim, ova odluka zavisi od konkretnog zadatka – na primer, u analizi sentimenta uzvičnik ili upitnik mogu nositi korisnu informaciju o tonu teksta [8].
- **Uklanjanje brojeva** – u zadacima fokusiranim na semantički sadržaj teksta, kao što je klasifikacija teme, brojevi su najčešće irelevantni i mogu se ukloniti. Međutim, u zadacima ekstrakcije informacija ili prepoznavanja entiteta, brojevi mogu biti ključni podaci.

Važno je naglasiti da je pojam šuma uvek relativan u odnosu na konkretni zadatak. Agresivno uklanjanje elemenata koji se na prvi pogled čine kao šum može dovesti do gubitka korisnih

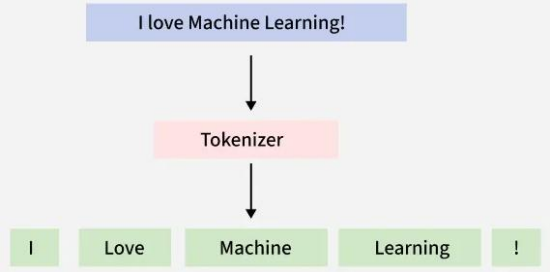
informacija i narušiti performanse modela. Zbog toga svaki korak uklanjanja šuma treba prilagoditi prirodi problema koji se rešava. Na slici 3.1. je prikazan jednostavan dijagram odluke šta je potrebno uraditi sa šumom. Crvene strelice označavaju uklanjanje, dok zelene označavaju zadržavanje šuma.



Slika 3.1. Dijagram odluke šta je potrebno uraditi sa šumom [5]

3.3. Tokenizacija

Tokenizacija je fundamentalni korak u obradi prirodnog jezika koji podrazumeva deljenje tekstualnog ulaza na manje jedinice poznate kao tokeni. Ovi tokeni mogu biti reči, karakteri, podreči ili rečenice. Tokenizacija oblikuje celokupni NLP tok obrade i predstavlja osnovu za sve naredne korake obrade teksta [9].



Slika 3.2. Primer tokenizacije [9]

Tokenizacija se može klasifikovati u nekoliko vrsta u zavisnosti od toga kako se tekst segmentira:

- **Tokenizacija na nivou reči** – najčešće korišćena metoda u kojoj se tekst deli na pojedinačne reči. Dobro funkcioniše za jezike sa jasnim granicama između reči. Na primer, rečenica “Mašinsko učenje je zanimljivo” postaje: [“Mašinsko”, “učenje”, “je”, “zanimljivo”].
- **Tokenizacija na nivou karaktera** – tekstualni podaci se dele na niz pojedinačnih karaktera. Na primer, reč “pomoć” postaje: [“p”, “o”, “m”, “o”, “ć”]. Korisna je za zadatke koji zahtevaju detaljnu analizu, kao što je korekcija pravopisa, ili za jezike sa nejasnim granicama između reči.

- **Tokenizacija na nivou podreči (engl. *subword*)** – predstavlja kompromis između tokenizacije na nivou reči i karaktera, deleći tekst na jedinice veće od pojedinačnog karaktera, ali manje od cele reči. Na primer, reč "nepredvidivost" može biti podeljena na ["ne", "predvidiv", "ost"]. Posebno je korisna za morfološki bogate jezike i rukovanje rečima van vokabulara. Koriste je savremeni modeli poput BERT-a kroz algoritme kao što su BPE (engl. *Byte-pair encoding*) i *WordPiece*.
- **Tokenizacija na nivou rečenice** – paragraf ili duži tekst se deli na pojedinačne rečenice kao tokene. Na primer, tekst "Veštačka inteligencija je u razvoju. Mašinsko učenje je njena grana." postaje: ["Veštačka inteligencija je u razvoju.", "Mašinsko učenje je njena grana."]
- **N-gram tokenizacija** – tekst se deli na sekvence od *n* uzastopnih reči. Na primer, tokenizacija bigramima rečenice "Obrada prirodnog jezika je složena" daje: [("Obrada", "prirodnog"), ("prirodnog", "jezika"), ("jezika", "je"), ("je", "složena")].

Tokenizacija je važna iz više razloga. Pre svega, omogućava efikasnu obradu teksta tako što sirovi tekst svodi na manje jedinice pogodne za statističku i računarsku analizu. Tokeni se zatim koriste kao karakteristike u modelima mašinskog učenja, čime se tekstualni podaci prevode u numerički oblik koji algoritmi mogu da obrađuju. Pored toga, tokenizacija je neophodna za pretraživanje informacija – indeksiranje i pretraga u sistemima za skladištenje podataka zasnovani su upravo na tokenima. U zadacima poput analize sentimenta i prepoznavanja imenovanih entiteta, tokenizacija omogućava određivanje funkcije i konteksta pojedinačnih reči. Konačno, tokenizacija pomaže u upravljanju vokabularom korpusa i prilagođava se potrebama konkretnog zadatka, kao što su sumarizacija teksta ili mašinsko prevođenje.

3.4. Normalizacija teksta

Normalizacija teksta je skup tehnika kojima se tekstualni podaci prevode u standardizovani, konzistentan oblik pre daljeg procesiranja. Cilj normalizacije nije uklanjanje šuma, već ujednačavanje različitih oblika iste reči ili istog koncepta kako bi ih model tretirao kao identične. Na primer, reči "Računarstvo", "računarstvo" i "RAČUNARSTVO" semantički su identične, ali model koji nije prošao kroz normalizaciju tretiraće ih kao tri potpuno različita tokena. Bez normalizacije, vokabular se nepotrebno povećava, kvalitet naučenih reprezentacija opada, a sposobnost generalizacije modela se smanjuje [10].

3.4.1. Normalizacija veličine slova (*case folding*)

Case folding je jedan od najjednostavnijih i najčešće primenjivanih koraka normalizacije, koji najčešće podrazumeva prevođenje svih karaktera u tekstu u mala slova. Na taj način reči koje se razlikuju samo po veličini slova tretiraju se kao isti token. Ovaj korak direktno smanjuje veličinu vokabulara i poboljšava konzistentnost reprezentacije teksta.

Važno je napomenuti razliku između prostog prevođenja u mala slova i pravog *case folding*-a. Prosto prevođenje u mala slova može dati nekonzistentne rezultate za određene jezike – na primer, nemački karakter "ß" prevodi se u velika slova kao "SS", ali "SS" prevođenjem u mala slova daje "ss", a ne originalno "ß". *Case folding*, implementiran kroz *casefold()* metodu u *Python*-u, rešava ovakve jezičke specifičnosti i osigurava da sve varijante iste reči budu svedene na isti kanonski oblik, što ga čini pouzdanijim izborom za NLP zadatke.

Međutim, *case folding* nije uvek pogodan. U nekim zadacima veliko slovo nosi semantičku informaciju – na primer, "*Apple*" kao naziv kompanije i "*apple*" kao voće su različiti koncepti. U takvim slučajevima, posebno u zadacima prepoznavanja imenovanih entiteta, *case folding* može dovesti do gubitka korisnih informacija i treba ga primeniti selektivno.

3.4.2. *Unicode* normalizacija

Unicode je međunarodni standard za kodiranje karaktera koji obuhvata znakove gotovo svih pisama i simbola na svetu. Iako definiše jedinstveni skup karaktera, isti vizuelni karakter često može biti predstavljen na više različitih načina u binarnom obliku. Na primer, karakter "é" može biti kodiran kao jedan jedinstven *Unicode* karakter (U+00E9), ali i kao kombinacija karaktera "e" (U+0065) i dijakritičkog znaka (U+0301). Za čoveka ova dva zapisa izgledaju identično, ali ih računar tretira kao različite tokene, što nepotrebno povećava vokabular i može negativno uticati na kvalitet modela. Da bi se ovaj problem rešio, Unicode definiše četiri standardna oblika normalizacije [11]:

- **NFC (kanonska kompozicija)** – tekst se prevodi u najkompaktniji oblik kombinovanjem osnovnog karaktera i dijakritičkog znaka u jedan kompozitni karakter. Na primer, "e" + "◌́" postaje "é". Ovo je najčešće korišćen oblik u praksi jer proizvodi kompaktnu reprezentaciju koja odgovara onome što korisnici očekuju.
- **NFD (kanonska dekompozicija)** – suprotan proces od NFC. Svaki kompozitni karakter se razlaže na osnovni karakter i kombinujuće znakove. Korisno kada je potrebno manipulisati dijakritičkim znacima zasebno, na primer pri njihovom uklanjanju.
- **NFKD (kompatibilna dekompozicija)** – pored kanonske dekompozicije, primenjuje i mapiranja kojima se vizuelno slični ali tehnički različiti karakteri svode na standardni oblik. Na primer, ligatura "fi" se rastavlja na "f" i "i", rimski broj "IV" postaje "IV", a cifra u krugu "①" postaje "1".
- **NFKC (kompatibilna dekompozicija praćena kanonskom kompozicijom)** – kombinuje kompatibilnu dekompoziciju sa ponovnom kompozicijom karaktera. Ovo je najagresivniji oblik normalizacije i najčešće se preporučuje za NLP zadatke jer obezbeđuje i standardizaciju oblika i kompoziciju karaktera, što rezultuje konzistentnim tokenima bez varijanti.

Unicode normalizacija je posebno važna pri radu sa višejezičnim skupovima podataka i tekstom prikupljenim sa *web*-a, gde različiti izvori mogu koristiti različite načine kodiranja

istih karaktera. Bez ovog koraka, model može naučiti različite reprezentacije za ono što je suštinski ista reč, što direktno narušava kvalitet ekstrakcije karakteristika.

3.4.3. Proširivanje skraćenica

Skraćenice su skraćeni oblici reči ili fraza koji se često pojavljuju u neformalnom tekstu – posebno u porukama, objavama i komentarima na društvenim mrežama i potrošačkim žalbama. Primeri uključuju oblike poput "*I'm*", "*don't*", "*can't*" ili neformalne oblike kao što su "*gonna*" i "*wanna*". Iako su po značenju identični sa svojim punim oblicima, tokenizator ih tretira kao potpuno različite tokene, što nepotrebno povećava vokabular i otežava modelu da nauči konzistentne reprezentacije istih koncepata. Razlikuju se nekoliko vrsta skraćenica [12]:

- **Glagolske skraćenice** – najčešća vrsta u engleskom jeziku, nastale spajanjem pomoćnog glagola sa negacijom ili zamenicom (*I'm → I am*, *don't → do not*, *can't → cannot*),
- **Neformalne skraćenice sa društvenih mreža** – skraćeni oblici karakteristični za neformalni pisani jezik (*gonna → going to*, *wanna → want to*, *gotta → got to*),
- **Skraćenice specifične za domen** – skraćeni oblici karakteristični za određenu oblast, kao što su medicinske, pravne ili finansijske skraćenice.

Proširivanje skraćenica se najčešće implementira pomoću unapred definisanog rečnika koji mapira skraćene oblike na njihove pune ekvivalente. U *Python*-u se za ovu svrhu često koristi biblioteka *contractions* koja sadrži sveobuhvatan rečnik engleskih skraćenica i automatski ih proširuje.

Treba napomenuti da neke skraćenice mogu biti višeznačne u zavisnosti od konteksta. Na primer, u engleskom jeziku, "*it's*" može biti prošireno kao "*it is*" ili "*it has*", a "*he'd*" kao "*he would*" ili "*he had*". Jednostavni rečnici mapiranja ne uzimaju kontekst u obzir, što može dovesti do pogrešnog proširivanja u određenim slučajevima.

3.4.4. Normalizacija *emoji* simbola

Emoji su vizuelni simboli koji se široko koriste u neformalnoj pisanoj komunikaciji i mogu nositi značajnu semantičku informaciju – posebno u pogledu emocija i tona teksta. Međutim, standardni tokenizatori najčešće nisu prilagođeni za njihovu obradu i tretiraju ih kao nepoznate karaktere, što može dovesti do gubitka informacija ili uvođenja šuma u reprezentaciju teksta. Pored toga, isti *emoji* može biti kodiran na više različitih načina u zavisnosti od *Unicode* varijante i platforme, što dodatno komplikuje konzistentnu obradu. Postoje tri osnovna pristupa normalizaciji *emoji* simbola, a izbor zavisi od prirode zadatka [13]:

- **Potpuno uklanjanje** – *emoji* se uklanjaju iz teksta. Ovaj pristup je prikladan kada *emoji* ne doprinose zadatku koji se rešava, ili kada je skup podataka pretežno formalan i *emoji* su retki.
- **Zamena *placeholder* tokenom** – svi *emoji* se zamenjuju jedinstvenim tokenom kao što je na primer "*EMOJI*". Ovaj pristup zadržava informaciju o tome da je na određenom mestu u tekstu postojao *emoji*, bez uvođenja velikog broja retkih tokena u vokabular.

- **Zamena tekstualnim opisom** – *emoji* se prevode u tekstualni opis njihovog značenja, na primer 😊 postaje "*smiling face*". Ovaj pristup je najinformativniji jer zadržava semantičku vrednost *emoji* simbola i posebno je koristan u zadacima analize sentimenta.

Odluka o tome koji pristup primeniti direktno zavisi od zadatka. U analizi sentimenta i analizi tekstova sa društvenih mreža, *emoji* često direktno izražavaju emociju i njihovo uklanjanje može značajno narušiti performanse modela. U zadacima klasifikacije formalnih tekstova, kao što su pravni ili medicinski dokumenti, *emoji* su uglavnom irelevantni i mogu se ukloniti. Za implementaciju u *Python*-u najčešće se koristi biblioteka *emoji*, koja omogućava detekciju i jednostavnu konverziju *emoji* simbola u njihove tekstualne opise.

3.4.5. Korekcija pravopisa

Korekcija pravopisa je korak predobrade koji podrazumeva automatsku detekciju i ispravku tipografskih grešaka u tekstu. Pogrešno napisane reči predstavljaju poseban problem u NLP *pipeline*-u jer ih model tretira kao potpuno nove, nepoznate tokene – na primer, "receive", "receve" i "receive" biće tretirane kao tri različite reči iako predstavljaju isti koncept. Posledica je nepotrebno povećanje vokabulara, degradacija kvaliteta naučenih reprezentacija i smanjenje tačnosti modela u zadacima kao što su klasifikacija teksta, analiza sentimenta i mašinsko prevođenje.

Većina algoritama za korekciju pravopisa zasnovana je na konceptu rastojanja izmene (engl. *edit distance*) – minimalnog broja operacija umetanja, brisanja ili zamene karaktera potrebnih da se jedna reč transformiše u drugu. Na osnovu ovog rastojanja i frekvencija reči u unapred izračunatom rečniku, algoritam predlaže najverovantniji ispravni oblik pogrešno napisane reči.

Treba napomenuti da korekcija pravopisa nije uvek bezbedna operacija. Domenski specifični termini, akronimi i vlastita imena često nisu zastupljeni u standardnim rečnicima i mogu biti pogrešno ispravljeni u uobičajene reči. Pored toga, korekcija bez uzimanja konteksta u obzir može zameniti ispravno napisanu reč njenim bliskim, ali semantički različitim ekvivalentom. Zbog toga je pri primeni korekcije pravopisa preporučljivo koristiti domenske rečnike i pažljivo proceniti da li je ovaj korak opravdan za konkretan skup podataka [14].

3.5. Uklanjanje stop reči

Stop reči su visokofrekventne reči koje nose malo ili nimalo semantičke vrednosti u kontekstu analize teksta. To su pre svega funkcionalne reči poput članova, predloga i veznika – na primer "i", "je", "se", "u", "na" – koje se pojavljuju u gotovo svakom tekstu bez obzira na temu ili sadržaj. Njihovo uklanjanje omogućava modelu da se fokusira na sadržajne reči koje zaista razlikuju jedan tekst od drugog [15].

Razlikuju se tri osnovne kategorije stop reči:

- **Standardne stop reči** – opšte funkcionalne reči kao što su veznici i predlozi, koje su zastupljene u unapred definisanim listama biblioteka poput NLTK-a (za engleski jezik),
- **Domenski specifične stop reči** – reči koje su visokofrekventne u određenoj oblasti, ali ne doprinose klasifikaciji; na primer, reč "pacijent" u medicinskim tekstovima može biti toliko učestala da ne razlikuje pojedine kategorije,
- **Kontekstualne stop reči** – reči sa ekstremno visokom frekvencijom u konkretnom skupu podataka, koje se identifikuju analizom distribucije frekvencija i dodaju u listu stop reči.

Uklanjanje stop reči donosi nekoliko prednosti – smanjuje dimenzionalnost vektora reprezentacije, poboljšava računarsku efikasnost i fokusira model na semantički relevantne tokene. Zadaci kod kojih je uklanjanje stop reči poželjno su:

- **Klasifikacija teksta i analiza sentimenta**,
- **Pretraživanje informacija i pretraživači**,
- **Modelovanje tema i klasterovanje**,
- **Ekstrakcija ključnih reči**.

Međutim, postoje i zadaci u kojima stop reči nisu šum već nosioci važnih informacija:

- **Mašinsko prevođenje** – stop reči čuvaju gramatičku strukturu rečenice koja je neophodna za ispravan prevod,
- **Sumarizacija teksta** – uklanjanje narušava koherentnost rečenica,
- **Sistemi za odgovaranje na pitanja** – sintaksičke veze između reči su od ključnog značaja.

Standardne liste stop reči mogu biti proširene domenski specifičnim terminima koji se pojavljuju previše često u konkretnom skupu podataka, ali ne doprinose klasifikaciji. Ovaj pristup se implementira analizom frekvencija reči u skupu podataka i dodavanjem reči koje prelaze određeni prag učestalosti u prilagođenu listu stop reči. Međutim, previše agresivno uklanjanje stop reči može dovesti do gubitka korisnih signala i narušiti performanse modela, zbog čega ovaj korak zahteva pažljivo prilagođavanje karakteristikama konkretnog zadatka.

3.6. Lematizacija i stemovanje

Lematizacija i stemovanje su tehnike normalizacije teksta čiji je cilj svođenje različitih oblika iste reči na zajednički osnovni oblik, čime se smanjuje veličina vokabulara i poboljšava konzistentnost reprezentacije teksta. Iako dele isti cilj, ove dve tehnike se značajno razlikuju u pristupu, preciznosti i računarskoj složenosti [16].

3.6.1. Stemovanje

Stemovanje je tehnika zasnovana na pravilima koja mehanički odseca prefikse i sufikse reči kako bi dobila njen koren. Algoritam ne uzima u obzir kontekst niti vrši lingvističku validaciju – svaka reč se obrađuje zasebno primenom unapred definisanih heurističkih pravila. Rezultat ovog procesa naziva se stem i ne mora biti validna reč u jeziku. Na primer, reč "studies" stemovanjem daje "studi", a "communication" daje "commun" – oblici koji ne postoje u rečniku, ali grupišu morfološki srodne reči pod isti koren.

Najpoznatiji algoritmi za stemovanje su *Porter Stemmer*, *Snowball* i *Lancaster*, koji se razlikuju po agresivnosti odsecanja. Glavna prednost stemovanja je brzina i računarska efikasnost, što ga čini pogodnim za primenu na velikim skupovima podataka. Nedostatak je manja preciznost – agresivno odsecanje može grupisati semantički različite reči pod isti stem ili proizvesti oblike koji otežavaju interpretaciju rezultata.

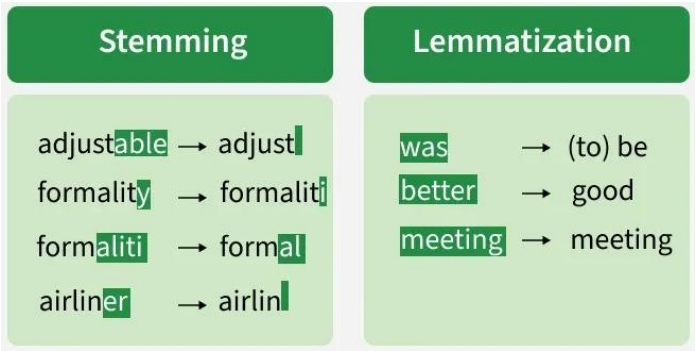
3.6.2. Lematizacija

Lematizacija je lingvistički pristup normalizaciji koji svodi reč na njen kanonski oblik poznat kao lema, uvažavajući gramatički kontekst i morfološka pravila jezika. Za razliku od stemovanja, lematizacija uvek proizvodi validnu reč koja postoji u rečniku. Na primer, "running" → "run", "better" → "good", "was" → "be".

Proces lematizacije obuhvata tri koraka: određivanje vrste reči (POS tagovanje), pretragu leksičke baze podataka i primenu morfoloških pravila. Uzimanje vrste reči u obzir je ključno – ista reč može imati različite leme u zavisnosti od toga da li se koristi kao imenica, glagol ili pridev. Na primer, "studies" kao imenica daje lemu "study", dok bi kao glagol takođe dalo "study", ali uz drugačiju morfološku analizu. Lematizacija je računarski zahtevnija od stemovanja, ali proizvodi semantički preciznije i interpretabilnije rezultate.

3.6.3. Poređenje tehnika

Na slici 3.3. je data ilustracija razlike između stemovanja i lematizacije. Može se videti da stemovanje često daje reči koje nisu validne.



Slika 3.3. Stemovanje vs. lematizacija [16]

Izbor između stemovanja i lematizacije zavisi od konkretnog zadatka i dostupnih resursa:

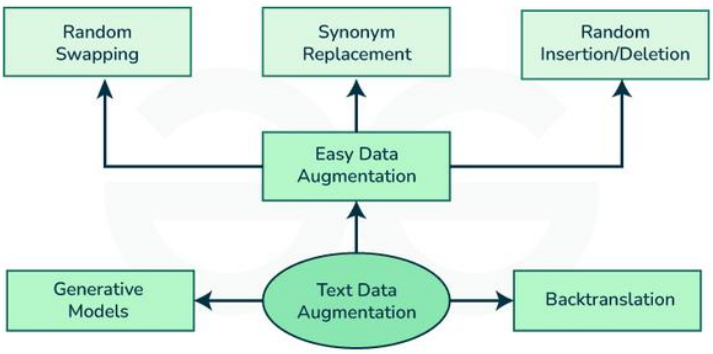
- **Stemovanje** je pogodno kada je brzina obrade važnija od preciznosti – tipično u pretraživanju informacija, indeksiranju dokumenata i sistemima za podudaranje ključnih reči gde je cilj povećanje odziva pretrage.
- **Lematizacija** je pogodna kada semantička preciznost ima prioritet – u analizi sentimenta, modelovanju tema, klasifikaciji teksta i sistemima za odgovaranje na pitanja gde pogrešno grupisanje reči može narušiti performanse modela.

U savremenim NLP *pipeline*-ovima za klasifikaciju teksta lematizacija je generalno preferiran pristup, dok stemovanje ostaje relevantan izbor u sistemima sa ograničenim računarskim resursima ili kada se radi sa izuzetno velikim skupovima podataka.

3.7. Augmentacija tekstualnih podataka

Augmentacija tekstualnih podataka je tehnika kojom se veštački proširuje skup podataka za treniranje kreiranjem novih, izmenjenih verzija postojećih tekstova bez potrebe za dodatnim prikupljanjem podataka. Cilj augmentacije je poboljšanje performansi i generalizacije modela povećanjem količine i raznovrsnosti podataka. Augmentacija je posebno korisna u situacijama kada je ukupna količina podataka nedovoljna za treniranje modela, kada postoji značajna nebalansiranost između klasa ili kada skup podataka nema dovoljno jezičke raznovrsnosti.

Augmentacija teksta može se primeniti na tri nivoa. Na **nivou reči** vrši se transformacija ili zamena pojedinačnih reči unutar teksta, kao što su zamena sinonimima ili nasumično brisanje. Na **nivou rečenice** menjaju se struktura i kompozicija celih rečenica kroz tehnike poput parafraziranja ili nasumičnog menjanja redosleda rečenica. Na **nivou dokumenta** augmentacija se primenjuje na ceo tekst i može uključivati umetanje ili uklanjanje paragrafa, promenu redosleda sekcija ili izmenu stila pisanja. Na slici 3.4. prikazana je podela tehnika za augmentaciju tekstualnih podataka [17].



Slika 3.4. Podela tehnika za augmentaciju tekstualnih podataka [17]

3.7.1. Jednostavna augmentacija tekstualnih podataka

Jednostavna augmentacija obuhvata grupu tehnika augmentacije na nivou reči koje su računarski efikasne i lako primenjive:

- **Zamena sinonimima** – nasumično odabrane reči u tekstu zamenjuju se sinonimima iz leksičke baze podataka kao što je *WordNet* ili pronalaženjem najbližih reči u prostoru *word embedding*-a.
- **Nasumična zamena** – dve nasumično odabrane reči u rečenici međusobno menjaju mesta, čime se uvodi varijabilnost u redosledu reči uz očuvanje semantičkog sadržaja.
- **Nasumično umetanje i brisanje** – nasumično se umeću semantički slične reči na proizvoljne pozicije u tekstu, ili se brišu redundantne reči koje ne menjaju značenje rečenice.

3.7.2. Augmentacija prevođenjem

Augmentacija prevođenjem (engl. *back translation*) je tehnika u kojoj se originalni tekst prevodi na jedan ili više drugih jezika, a zatim nazad na originalni jezik. Budući da mašinski prevod retko kada reprodukuje identičan tekst, rezultat je prirodna parafraza originalne rečenice koja uvodi leksičku i sintaksičku raznovrsnost. Na primer, prevođenje sa engleskog na francuski i nazad na engleski daje drugačiju formulaciju iste misli. Glavni nedostatak ove tehnike je da se pri prevodu mogu izgubiti nijanse i specifični termini, posebno kod domenski specifičnih tekstova.

3.7.3. Generativni modeli

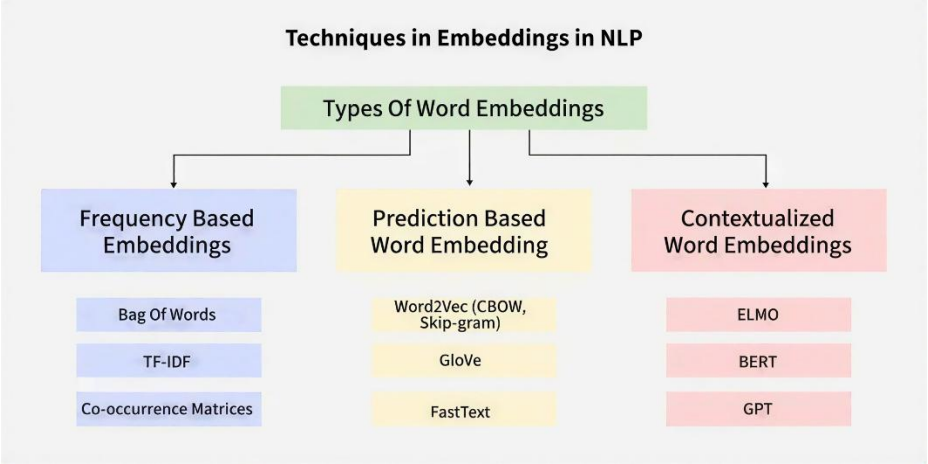
Najnapredniji pristup augmentaciji teksta podrazumeva korišćenje velikih jezičkih modela (LLM) za generisanje semantički ekvivalentnih tekstova. Parafraziranje pomoću LLM-ova omogućava kreiranje raznovrsnih verzija originalnog teksta koje zadržavaju puno značenje uz značajne leksičke i sintaksičke razlike. Ovaj pristup daje kvalitetnije rezultate od jednostavnih tehnika, ali zahteva više računarskih resursa.

3.7.4. Prednosti i mane augmentacije tekstualnih podataka

Augmentacija podataka donosi nekoliko ključnih prednosti – povećava raznovrsnost skupa podataka, poboljšava generalizaciju modela i pomaže u rešavanju problema nebalansiranosti klasa. Međutim, postoje i važna ograničenja kojih treba biti svestan. Augmentovane instance ne smeju biti previše slične originalnim podacima, jer to može dovesti do preučenja, ali ni previše različite, jer model neće naučiti relevantne obrasce. Pored toga, postoji rizik od uvođenja nenamerne pristrasnosti u podatke, a augmentacija povećava računarsku složenost i vreme treniranja. Zbog toga augmentaciju treba primenjivati selektivno i uvek pratiti njen uticaj na performanse modela.

4. UTICAJ PREDOBRADE NA METODE REPREZENTACIJE TEKSTA

Tehnike reprezentacije teksta prevode sirove tekstualne podatke u numerički oblik koji modeli mašinskog učenja mogu da obrađuju. Međutim, različite tehnike reprezentacije oslanjaju se na fundamentalno različite pretpostavke o tome šta čini smislenu jedinicu teksta – neke tretiraju svaku reč kao nezavisan token, druge uče semantičke odnose između reči, a treće modeluju čitav kontekst rečenice odjednom. Zbog tih razlika, iste odluke donete u fazi predobrade mogu imati dijametralno suprotan efekat na različite tehnike – korak koji poboljšava performanse jednog modela može biti neutralan ili čak štetan za drugi. Na slici 4.1. je prikazana podela tehnika za reprezentaciju teksta.



Slika 4.1. Podela tehnika za reprezentaciju teksta [18]

U nastavku ovog poglavlja analizira se kako predobrada teksta utiče na tri grupe tehnika reprezentacije: klasične metode zasnovane na frekvencijama reči (BoW i TF-IDF), metode zasnovane na vektorskim reprezentacijama reči (*Word2Vec* i *FastText*) i savremene kontekstualne modele zasnovane na *transformer* arhitekturi (BERT i srodni modeli).

4.1. Klasične metode zasnovane na frekvencijama reči (BoW i TF-IDF)

Bag of Words (BoW) i *Term Frequency-Inverse Document Frequency* (TF-IDF) su klasične tehnike reprezentacije teksta koje prevode tekstualne dokumente u numeričke vektore na osnovu frekvencija reči. Kod BoW modela, svaki dokument se predstavlja kao vektor čija dužina odgovara broju reči u vokabularu, a vrednosti predstavljaju broj pojavljivanja svake reči u dokumentu. Redosled reči se pri tome potpuno ignoriše – odatle i naziv "vreća reči".

TF-IDF je nadogradnja BoW modela koja uzima u obzir ne samo frekvenciju reči u dokumentu, već i njenu učestalost kroz ceo skup podataka. Reči koje se pojavljuju u svim

dokumentima dobijaju nisku težinu, dok reči specifične za manji broj dokumenata dobijaju višu težinu. TF-IDF skor reči u dokumentu računa se kao:

$$TF - IDF(t, d) = TF(t, d) \times IDF(t)$$

gde je:

$$TF(t, d) = \frac{broj\ pojavljivanja\ reči\ t\ u\ dokumentu\ d}{ukupan\ broj\ reči\ u\ dokumentu\ d}$$

$$IDF(t) = log\frac{ukupan\ broj\ dokumenata}{broj\ dokumenata\ koji\ sadrže\ reč\ t}$$

Obe metode su zasnovane na vokabularu celog skupa podataka, što ih čini posebno osetljivim na kvalitet predobrade. Svaka jedinstvena reč u vokabularu direktno dodaje jednu dimenziju vektoru reprezentacije, pa gotovo svaki korak predobrade ima direktan uticaj na dimenzionalnost i gustinu matrice reprezentacije.

Normalizacija teksta ima direktan uticaj na veličinu vokabulara. Bez *case folding*-a, reči "Banka", "banka" i "BANKA" tretiraju se kao tri različita tokena i generišu tri zasebne dimenzije u matrici, iako predstavljaju isti koncept. Slično, bez *Unicode* normalizacije različita kodiranja istog karaktera mogu rezultovati duplikatima u vokabularu. Proširivanje skraćenica sprečava dupliranje istog koncepta pod različitim oblicima – na primer, "I'm" i "I am" bez proširivanja postaju dva različita tokena.

Lematizacija i stemovanje dodatno smanjuju vokabular grupisanjem morfoloških varijanti iste reči pod jedan zajednički token. Na primer, bez lematizacije reči "trčanje", "trčao" i "trčati" generišu tri zasebne dimenzije, dok lematizacijom sve postaju jedan token. Direktna posledica je povećanje frekvencije relevantnih termina, što poboljšava TF-IDF težine jer model ima više primera po tokenu i pouzdanije procenjuje njegovu važnost.

Uklanjanje stop reči ima posebno izražen efekat na TF-IDF reprezentaciju. Stop reči pojavljuju se u gotovo svim dokumentima, što im daje visoku vrednost TF, ali istovremeno i veoma nizak IDF rezultat, jer $\log\frac{N}{df}$ teži nuli kada df teži N. Rezultat je zanemarljivo mali TF-IDF skor koji ne doprinosi klasifikaciji. Njihovim uklanjanjem matrica postaje kompaktnija i fokusirana isključivo na sadržajne reči koje zaista razlikuju jednu klasu od druge.

Poseban problem predstavljaju domenski specifični termini koji su visokofrekventni unutar skupa podataka, ali nisu prepoznati kao standardne stop reči. Takvi termini mogu dobiti visoke TF-IDF vrednosti i dominirati reprezentacijom, iako ne doprinose razlikovanju klasa. Rešenje je proširivanje standardnih lista stop reči domenski specifičnim terminima identifikovanim analizom frekvencija u konkretnom skupu podataka [19].

4.2. Vektorske reprezentacije reči (*Word2Vec*, *GloVe* i *FastText*)

Vektorske reprezentacije reči (engl. *word embeddings*) su guste vektorske reprezentacije koje, za razliku od BoW i TF-IDF, hvataju semantičke odnose između reči. Svaka reč se predstavlja kao vektor realnih brojeva u kontinualnom vektorskom prostoru, pri čemu su semantički slične reči blizu jedna drugoj. Na primer, vektori reči "kralj" i "kraljica" biće bliži jedni drugima nego vektorima reči "automobil" ili "banka". *Word2Vec* uči ove reprezentacije treniranjem na velikom korpusu tekstova koristeći dva pristupa – CBOW, koji predviđa reč na osnovu konteksta, i *Skip-Gram*, koji predviđa kontekst na osnovu reči. *GloVe* predstavlja alternativni pristup koji uči reprezentacije na osnovu globalne matrice ko-pojavljivanja reči umesto lokalnog konteksta, ali deli iste karakteristike u pogledu osetljivosti na predobradu kao *Word2Vec*, tako da neće biti posebno analiziran. *FastText* proširuje ovaj pristup radom na nivou podreči, o čemu će biti više reči u nastavku.

Za razliku od BoW i TF-IDF koji se grade nad statičkim vokabularom, *word embeddings* se uče iz samog teksta, što znači da kvalitet predobrade direktno određuje kvalitet naučenih vektorskih reprezentacija. Šum u tekstu, morfološke varijante i nedoslednosti u normalizaciji uvode lažne statističke obrasce koje model uči kao da su smisleni. Na primer, ako korpus sadrži i "trčati" i "trči" i "trčanje" kao zasebne reči bez lematizacije, model će pokušati da nauči zasebne vektorske reprezentacije za svaki od tih oblika, rasipajući kapacitet na morfološke varijante umesto na semantičke odnose.

Jedan od ključnih problema kod vektorskih reprezentacija je pojava reči van vokabulara (engl. *Out-of-Vocabulary*, OOV) – reči koje se pojavljuju u test skupu ali nisu viđene tokom treniranja. Model nema mogućnost da dodeli *embedding* vektor takvim rečima, što može značajno narušiti performanse. Lematizacija i stemovanje smanjuju OOV problem jer grupisanjem morfoloških varijanti povećavaju verovatnoću da će svaka reč biti viđena tokom treniranja u svom kanonskom obliku. Umesto da model mora da nauči zasebne vektore za različite reči sa istom osnovom, lematizacija ih svodi na jednu osnovnu reč čime se povećava frekvencija tog tokena u korpusu i poboljšava kvalitet naučenog vektora.

Word2Vec i *FastText* se značajno razlikuju u osetljivosti na predobradu. *Word2Vec* tretira svaku reč kao atomsku jedinicu i ne može generisati reprezentacije za reči koje nisu viđene tokom treniranja. *FastText*, s druge strane, predstavlja svaku reč kao skup n-grama karaktera – na primer, reč "banka" se predstavlja kao skup podreči "<ba", "ban", "ank", "nka", "ka>". Zahvaljujući ovom pristupu, *FastText* može generisati *embedding* vektore za nepoznate reči kombinovanjem vektora njihovih podreči, što ga čini znatno robusnijim na OOV problem i manje zavisnim od agresivne predobrade. Kod *Word2Vec* modela, lematizacija i korekcija pravopisa su kritičniji koraci predobrade, dok *FastText* može u određenoj meri kompenzovati njihovo izostavljanje.

Augmentacija podataka ima direktan pozitivan uticaj na kvalitet *word embedding*-a. Veći i raznovrsniji korpus omogućava modelu da nauči preciznije i stabilnije vektorske reprezentacije, posebno za reči koje se retko pojavljuju u originalnom skupu podataka. Tehnike poput zamene sinonimima i povratnog prevoda povećavaju broj konteksta u kojima se svaka reč pojavljuje, što direktno poboljšava kvalitet naučenih semantičkih odnosa između reči [20].

4.3. Transformeri (BERT i srodni modeli)

BERT (engl. *Bidirectional Encoder Representations from Transformers*) je kontekstualni jezički model zasnovan na *transformer* arhitekturi koji predstavlja kvalitativni skok u odnosu na prethodne tehnike reprezentacije teksta. Za razliku od *Word2Vec* i *FastText* koji svakoj reči dodeljuju jedan fiksni vektor bez obzira na kontekst, BERT generiše dinamičke reprezentacije koje zavise od celokupnog okruženja reči u rečenici – uzimajući u obzir istovremeno i levi i desni kontekst. Na primer, reč "list" u rečenicama "Pročitao sam svaki list knjige" i "Jesen je, svaki list pada sa drveta" dobija dva različita vektora koji odražavaju različita značenja.

BERT koristi sopstvenu tokenizaciju zasnovanu na *WordPiece* algoritmu – vrsti *subword* tokenizacije koja deli nepoznate reči na poznate podreči. Na primer, reč "tokenizacija" može biti tokenizovana kao ["token", "##iza", "##cija"]. Ovaj pristup praktično eliminiše problem reči van vokabulara koji je kritičan kod *Word2Vec* modela, jer model uvek može da reprezentuje nepoznatu reč kroz njene sastavne podreči. Direktna posledica je da BERT zahteva znatno manje agresivnu predobradu od klasičnih metoda – lematizacija, stemovanje i korekcija pravopisa nisu neophodni koraci kao kod BoW ili *Word2Vec*.

Ipak, predobrada nije potpuno irelevantna ni za *transformer*-e. BERT ima ograničen kontekstni prozor – BERT *Base* prihvata maksimalno 512 tokena po ulaznom tekstu. Prisustvo šuma u tekstu, kao što su HTML oznake, URL-ovi, ili višestruki nepotrebni razmaci, troši deo tog ograničenog prostora bez ikakve semantičke koristi, efektivno skraćujući korisni sadržaj koji model može da obradi. Zbog toga osnovno uklanjanje šuma ostaje preporučljiv korak čak i pri radu sa BERT modelima.

Međutim, određeni koraci predobrade koji poboljšavaju performanse BoW i *Word2Vec* modela mogu kod BERT-a biti neutralni ili čak kontraproduktivni. Lematizacija je konkretan primer – BERT je tokom treninga naučen da razume morfološke odnose između reči kao što su "trčanje", "trči" i "trčati", i te morfološke informacije koristi kao deo kontekstualnog razumevanja. Agresivnom lematizacijom pre ulaska u model te informacije se gube, čime se potencijalno narušava sposobnost modela da razlikuje glagolska vremena i oblike koji mogu biti semantički relevantni. Slično, uklanjanje stop reči koje su deo ustaljenih fraza može narušiti koherentnost ulaznog teksta i otežati modelu da prepozna poznate jezičke obrasce na kojima je treniran [21].

5. ZAKLJUČAK

Kvalitet tekstualnih podataka i proces njihove predobrade predstavljaju temelje na kojima počiva svaki sistem za klasifikaciju teksta zasnovan na mašinskom ili dubokom učenju. Kroz ovaj rad pokazano je da ni najsofisticiraniji model ne može kompenzovati loše pripremljene ulazne podatke – odluke donete u fazi predobrade direktno određuju kvalitet naučenih reprezentacija i konačne performanse modela.

U poglavlju o kvalitetu tekstualnih podataka analizirane su ključne mere kojima se procenjuje stanje sirovog teksta pre bilo kakve obrade – dužina dokumenata, veličina vokabulara, leksička raznovrsnost i prisustvo šuma. Ove mere nisu samo deskriptivne – one informišu sve naredne odluke u *pipeline*-u, od izbora tehnika predobrade do podešavanja arhitekture modela.

Poglavlje o predobradi tekstualnih podataka obuhvatilo je kompletan niz koraka koji transformišu sirovi tekst u oblik pogodan za treniranje modela. Svaki korak – od uklanjanja duplikata i šuma, kroz tokenizaciju i normalizaciju, do lematizacije i augmentacije – doprinosi smanjenju šuma, konzistentnosti reprezentacije i bogatstvu skupa podataka. Pri tome je naglašeno da ne postoji univerzalni recept: svaka odluka mora biti prilagođena karakteristikama konkretnog skupa podataka i zahtevima zadatka.

Analiza uticaja predobrade na metode reprezentacije teksta pokazala je da iste odluke mogu imati dijametralno suprotan efekat u zavisnosti od tehnike koja se koristi. Klasične metode poput BoW i TF-IDF su izuzetno osetljive na veličinu vokabulara i zahtevaju agresivnu predobradu, dok *word embedding* modeli poput *Word2Vec* i *FastText* zavise od kvaliteta i obima korpusa. *Transformer* modeli poput BERT-a su najmanje zavisni od predobrade zahvaljujući sopstvenoj *subword* tokenizaciji, ali ni oni nisu potpuno imuni – prisustvo šuma troši deo ograničenog kontekstnog prozora, a preterana predobrada može ukloniti morfološke informacije koje model koristi.

Razumevanje međuzavisnosti između kvaliteta podataka, tehnika predobrade i metoda reprezentacije teksta je ključno za izgradnju efikasnih NLP sistema, i upravo ta međuzavisnost bila je centralna tema ovog rada.

LITERATURA

[1] T. Alexakis, E. Adamopoulou, N. Peppes, E. Daskalakis and G. Ntouskas, "Evaluating Data Quality: Comparative Insights on Standards, Methodologies, and Modern Software Tools," MDPI, 30 July 2025. [Online]. Available: https://www.mdpi.com/2079-9292/14/15/3038. [Accessed 28 February 2026].

[2] P. Singh, "Why long documents pose a real challenge – and why embeddings still matter," Medium, 11 December 2025. [Online]. Available: https://medium.com/@PriyaSingh325/why-long-documents-pose-a-real-challenge-and-why-embeddings-still-matter-e06df6f1a539. [Accessed 1 March 2026].

[3] M. D. Jurafsky and J. H, "Speech and Language Processing," Stanford, 6 January 2026. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/. [Accessed 2 March 2026].

[4] "Lexical Density vs Lexical Diversity: What's the Difference?," TextInspector, 24 June 2022. [Online]. Available: https://textinspector.com/lexical-density-vs-lexical-diversity/. [Accessed 4 March 2026].

[5] "Handling Noise in Text Data," [Online]. Available: https://apxml.com/courses/nlp-fundamentals/chapter-1-nlp-text-processing-techniques/handling-text-noise. [Accessed 4 March 2026].

[6] "Text Data Duplicates," [Online]. Available: https://docs.deepchecks.com/stable/nlp/auto_checks/data_integrity/plot_text_duplicates.html. [Accessed 5 March 2026].

[7] "Find duplicates and near-duplicates in a corpus with Natural Language Processing," 3 October 2022. [Online]. Available: https://blogs.sas.com/content/subconsciousmusings/2022/10/03/find-duplicates-with-nlp/. [Accessed 5 March 2026].

[8] "How to remove punctuations in NLTK," GeeksForGeeks, 23 July 2025. [Online]. Available: https://www.geeksforgeeks.org/nlp/how-to-remove-punctuations-in-nltk/. [Accessed 6 March 2026].

[9] "Tokenization in NLP," GeeksForGeeks, 11 July 2025. [Online]. Available: https://www.geeksforgeeks.org/nlp/nlp-how-tokenizing-text-sentence-words-works/. [Accessed 5 March 2026].

[10] "Normalizing Textual Data with Python," GeeksForGeeks, 23 July 2025. [Online]. Available: https://www.geeksforgeeks.org/python/normalizing-textual-data-with-python/. [Accessed 6 March 2026].

[11] M. Brenndoerfer, "Text Normalization: Unicode Forms, Case Folding & Whitespace Handling for NLP," Michael Brenndoerfer, 19 March 2025. [Online]. Available: https://mbrenndoerfer.com/writing/text-normalization-unicode-nlp. [Accessed 7 March 2026].

[12] "How to deal with contractions in NLP," educative, [Online]. Available: https://www.educative.io/answers/how-to-deal-with-contractions-in-nlp. [Accessed 7 March 2026].

[13] E. Mousavi, "NLP Series: Day 5 — Handling Emojis: Strategies and Code Implementation," Medium, 4 March 2025. [Online]. Available: https://medium.com/@ebimsv/nlp-series-day-5-handling-emojis-strategies-and-code-implementation-0f8e77e3a25c. [Accessed 7 March 2026].

[14] "Spelling Correction in Text Preprocessing Using SymSpell," GeeksForGeeks, 23 July 2025. [Online]. Available: https://www.geeksforgeeks.org/nlp/spelling-correction-in-text-preprocessing-using-symspell/. [Accessed 7 March 2026].

[15] "Removing stop words with NLTK in Python," GeeksForGeeks, 7 October 2025. [Online]. Available: https://www.geeksforgeeks.org/nlp/removing-stop-words-nltk-python/. [Accessed 9 March 2026].

[16] "Lemmatization vs. Stemming," GeeksForGeeks, 5 January 2026. [Online]. Available: https://www.geeksforgeeks.org/nlp/lemmatization-vs-stemming/. [Accessed 9 March 2026].

[17] "Text augmentation techniques in NLP," GeeksForGeeks, 23 July 2025. [Online]. Available: https://www.geeksforgeeks.org/nlp/text-augmentation-techniques-in-nlp/. [Accessed 10 March 2026].

[18] "Word Embeddings in NLP," GeeksForGeeks, 23 July 2025. [Online]. Available: https://www.geeksforgeeks.org/nlp/word-embeddings-in-nlp/. [Accessed 13 March 2026].

[19] "Bag-of-words vs TF-IDF," GeeksForGeeks, 23 July 2025. [Online]. Available: https://www.geeksforgeeks.org/nlp/bag-of-words-vs-tf-idf/. [Accessed 13 March 2026].

[20] H. Vardhan, "A Comprehensive Guide to Word Embeddings in NLP," Medium, 11 July 2024. [Online]. Available: https://medium.com/@harsh.vardhan7695/a-comprehensive-guide-to-word-embeddings-in-nlp-ee3f9e4663ed. [Accessed 13 March 2026].

[21] "BERT Model - NLP," GeeksForGeeks, 11 September 2025. [Online]. Available: https://www.geeksforgeeks.org/nlp/explanation-of-bert-model-nlp/. [Accessed 13 March 2026].